Mastering the Python CLI

Executing Modules, Inline Scripts, and Essential Flags

The Power of -m

Running Packages as Built-in Scripts

What does -m actually mean?

The `-m` flag tells Python to look inside its internal library path paths and execute a library or module as a script.

- **Global Safety:** It explicitly binds execution to the specific version of Python you called.
- **No Path Guessing:** Avoids "command not found" shell path environment script issues.

Standard Anatomy:

```
python -m
```

Instead of hunting for an execution script location, Python handles the internal package resolutions cleanly.

Real-World -m Superpowers

Command Example	What It Instantly Does
`python -m venv .venv`	Launches the internal environment manager utility.
`python -m pip install django`	Runs `pip` safely bound to this exact Python binary block.
`python -m http.server 8000`	Spins up an instant local web server from your current folder!
`python -m json.tool data.json`	Validates and pretty-prints messy JSON files directly in terminal.

Essential CLI Flags

Controlling Interpreter State on the Fly

Executing Inline Code: -c

The `-c` (command) flag allows you to pass a short string of pure Python code directly into the terminal window for instant execution without building a `.py` file.

Great for quick testing, sanity checking mathematics, or fetching configuration metadata values.

Example Usage:

```
python -c "print('Hello from CLI')"
```

Checking Constants:

```
python -c "import sys; print(sys.path)"
```

Interactive Inspection: -i

When you run a standard Python file, the script completes and exits back out to your main shell prompt.

The `-i` flag forces Python to finish running the file, but keep the interpreter open so you can interactively audit existing variables and memory status functions.

Example Usage:

```
python -i script.py
```

💡 **Ideal for Debugging:**

If your script crashes near the bottom, running with `-i` drops you straight into a live terminal environment precisely where the execution stopped.

The "Silent" Flag: -O (Optimize)

The uppercase `-O` flag turns on basic code optimizations inside the execution engine scope.

- It completely ignores all statements beginning with `assert`.
- Helps run production scripts faster by bypassing testing checkpoints safely.

Example Usage:

```
python -O production_script.py
```

Checking Up: -V & --version

The simplest yet most useful diagnostics flag. Before running complex local scripts, confirm exactly what version profile is handling the execution loop.

Short Syntax

```
python -V
```

Long Syntax

```
python --version
```

The Complete Flags Reference Matrix

Flag	Name / Action	Ideal Target Scenario Use Case
`-m`	Run Module	Executing safe library operations like `pip` or `venv`.
`-c`	Run Command String	One-liner quick scripts without making dedicated files.
`-i`	Inspect Interactively	Dropping into shell mode after a file finishes running.
`-O`	Optimize Code	Stripping out safety assertions to increase runtime speed.
`-V`	Version Inquiry	Verifying interpreter environment specs before running apps.

Let's Combine Them!

Next, we will test running commands in the lab exercise manual below.